

EE5203 L10 Lab Guide

UART — make the board talk, and listen - Basri Erdogan

Mission: Stream timestamped ADC readings to your PC at 10 Hz over the ST-LINK virtual COM port, then accept typed commands that change the LED duty — with the tick, the PWM, and the ADC all still running underneath.

Prepare before class

- ☐ Re-read the theory slides through “Assembling a command.”
- ☐ Bring your working L09 project (tick + PWM + ADC).
- ☐ Compute by hand: BRR for 115 200 baud with PCLK1 = 16 MHz.
- ☐ Know what `\r\n` is for, and what TXE means that TC does not.

Learning objectives

By checkoff time, you can:

1. configure USART2 for 8N1 and verify BRR against your own arithmetic;
2. transmit formatted telemetry paced by the L07 tick, without blocking;
3. receive by interrupt, re-arm correctly, and act in `main()`;
4. explain TXE, TC, RXNE, and ORE from the SFR view.

Hardware and files

Item	Required value
Board	Nucleo-F401RE — start from a copy of the L09 project, saved as L10_UART
Wiring	none — PA2/PA3 are wired to the ST-LINK virtual COM port on the board
USART2	115 200 baud, 8 data bits, no parity, 1 stop, TX and RX
Monitor	VS Code Serial Monitor, 115 200 baud, line ending <code>\r\n</code>

Close the Serial Monitor before flashing. An open port blocks the programmer, and the error message does not mention the monitor.

Configure in CubeMX

1. **Connectivity** → **USART2**: Mode **Asynchronous**; 115 200 / 8 / None / 1.
2. Confirm PA2 and PA3 turn green — alternate function **AF7**.
3. **System Core** → **NVIC**: enable **USART2 global interrupt**.
4. Generate, then find the AFR assignment for PA2/PA3 in the generated GPIO init before writing any code of your own.

Checkpoint A - The board speaks

In USER CODE BEGIN 2, after `MX_USART2_UART_Init()`:

```
const char *banner = "EE5203 L10 ready\r\n";
HAL_UART_Transmit(&huart2, (uint8_t *)banner, strlen(banner), 100);
```

Evidence for Checkpoint A: the banner appears in the Serial Monitor at 115 200 baud — and turns to garbage if you reopen the monitor at 9600. Show both. Then, in the SFR view, read `USART2->BRR` and compare it against the value you computed before class.

Checkpoint B - Telemetry at 10 Hz

No `HAL_Delay()`. Schedule from the L07 tick:

```
if ((g_ms - t_tx) >= 100U) {
    t_tx = g_ms;
    uint16_t raw = adc_read(ADC_CHANNEL_0);
    uint32_t mv = (raw * 3300u) / 4095u;

    char buf[48];
    int n = snprintf(buf, sizeof buf, "%lu,%u,%lu\r\n",
                     (unsigned long)g_ms, (unsigned)raw, (unsigned long)mv);
    HAL_UART_Transmit(&huart2, (uint8_t *)buf, (uint16_t)n, 100);
}
```

Evidence for Checkpoint B: ten lines per second, timestamps 100 ms apart, values tracking the knob. The LED keeps breathing and the servo keeps responding throughout — the telemetry is not stealing the loop. Copy thirty lines into a spreadsheet and plot raw against time.

Checkpoint C - The board listens

Arm the receiver once, then re-arm inside the callback:

```
volatile uint8_t rx_byte;
volatile char    rx_ring[64];
volatile uint8_t head = 0, tail = 0;

HAL_UART_Receive_IT(&huart2, (uint8_t *)&rx_byte, 1);    /* arm once */

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART2) {
        rx_ring[head] = (char)rx_byte;
        head = (uint8_t)((head + 1) % 64);
        HAL_UART_Receive_IT(&huart2, (uint8_t *)&rx_byte, 1);    /* re-arm */
    }
}
```

Drain the ring in main(), assemble a line on \r or \n, and act:

Command	Effect
led on / led off	force the LED on or off
duty N	set the PWM duty, N in 0...999
anything else	reply ? unknown

Evidence for Checkpoint C: typed commands take effect immediately, a bad command is refused with a message, and telemetry keeps flowing while you type.

Checkpoint D - Break it on purpose

Evidence for Checkpoint D: set a breakpoint in main(), type several characters quickly while the core is halted, then resume. Show ORE set in USART2->SR, explain exactly which bytes were lost, and describe what the program must do to recover.

Individual extension

Pick one, predict first:

- **Non-blocking transmit:** replace HAL_UART_Transmit with HAL_UART_Transmit_IT and measure the difference with a GPIO pin toggled around the call.
- **Command with a reply:** add get which returns the current raw, mV, and duty on one line.
- **Baud sweep:** reconfigure to 9600 and to 230400, and record BRR, the actual baud, and the error at each. At which one does the telemetry rate stop fitting in the budget?

Acceptance checklist

- ☐ .ioc: USART2 asynchronous 8N1 at 115 200; PA2/PA3 AF7; USART2 NVIC on.
- ☐ BRR read from the SFR view matches your hand-computed value.
- ☐ Telemetry at a measured 10 Hz, snprintf (not sprintf), \r\n endings.
- ☐ Tick, PWM, and ADC all still working while telemetry streams.
- ☐ Receive is interrupt-driven and correctly re-armed; commands act in main().
- ☐ A bad command is answered, not ignored.
- ☐ ORE demonstrated deliberately and explained.

Individual oral check

- What is BRR for your baud, and what actual rate does it produce?
- Why 10 bit times for one byte, not 8?
- What is the difference between TXE and TC, and when does it matter?
- Why must HAL_UART_Receive_IT be called again inside the callback?
- Why is HAL_UART_Transmit acceptable in main() but not in a callback?
- Your monitor shows consistent garbage. Give the two most likely causes.

Submit

Submit `main.c`, thirty captured telemetry lines, and one SFR-view screenshot showing `USART2->BRR` and `USART2->SR` with `ORE` set — plus your hand computation of `BRR` alongside the register value.

If you are stuck

Nothing at all: monitor on the right COM port? Right baud? Was the monitor open while flashing? **Consistent garbage:** baud mismatch — check both ends. **Exactly one byte then silence:** the receive interrupt was never re-armed. **Everything freezes when you type:** you used blocking `HAL_UART_Receive`. **Text stair-steps down the screen:** `\n` without `\r`. **Reception dies after a burst:** `ORE` is set and must be cleared.